

Option Informatique MP, DS n°1

Algorithmique du texte

Note de début : ce DS porte sur un sujet de biologie. Les connaissances de l'auteur sur le sujet étant un peu approximative, veuillez ne pas utiliser ce DS comme source sur ce sujet.

Quand vous écrivez du code, il est attendu que celui-ci soit clair, commenté lorsque c'est nécessaire, et que les types d'entrée et de sortie des fonctions soient précisés.

On demande de ne pas utiliser les fonctions du module `String`.

Les questions suivies d'une étoile sont difficiles.

Si vous suspectez une erreur sur le sujet, expliquer votre raisonnement sur votre copie et éventuellement proposez une correction (ou bien juste passez la question).

I) Généralités

L'acide désoxyribonucléique, plus connu sous le nom d'ADN, est une molécule présente dans presque tout le vivant sur Terre. Elle contient toute l'information génétique, ou génome, d'un individu.

Les molécules d'ADN des cellules vivantes sont formées de deux brins antiparallèles enroulés l'un autour de l'autre pour former une double hélice.

Chaque brin d'ADN est constitué d'une série de nucléotides : adénine (A), cytosine (C), guanine (G) ou thymine (T).

Lorsqu'on travaille sur le génome d'un individu ou d'une espèce, on modélise en général le génome par une (très longue) chaîne de caractère, composée des 4 lettres suivantes : A, C, G, T.

Chez l'humain, l'ADN est composé d'environ 3 milliards de nucléotides. Il est donc nécessaire d'utiliser des algorithmes efficaces pour traiter ces chaînes de caractères.

II) Implémentation en OCaml

En langage OCaml, on modélise un génome par une liste de caractères :

```
type genome = char list
```

1. Définissez une variable OCaml pour modéliser le génome `AACGTTGCC`.

Dans les questions suivantes, on demande de définir des fonctions, souvent récursives, en précisant leur type. Vous devez annoter les arguments de vos fonctions avec leur types.

2. Définissez une fonction `compter` de type `genome -> char -> int` qui prend en argument un génome et l'identifiant d'un nucléotide (A, C, G, ou T), et qui renvoie le nombre d'occurrences de ce nucléotide dans le génome.

Dans une molécule d'ADN, les deux brins encodent en fait la même information : chaque nucléotide est situé en face de son complémentaire. Le complémentaire de A est T, et celui de C est G.

3. Donnez le complémentaire du brin `AACGTTGCC`.

4. Écrivez une fonction `complémentaire` de type `genome -> genome` qui prends un génome et renvoie son complémentaire.

III) Distance entre deux chaînes

En génétique, on peut s'intéresser au degré de différence entre les génomes, pour savoir à quel point deux individus, ou deux espèces, sont similaires.

III.1) Distance naïve

² On définit une première distance entre les chaînes de caractère de même longueur : la **distance naïve**.

Pour deux chaînes $A = [a_0; a_1; \dots; a_n]$ et $B = [b_0; b_1; \dots; b_n]$, la distance naïve entre A et B est :

$$d = \sum_{i=1}^n 1_{a_i=b_i}$$

où $1_{a_i=b_i}$ vaut 1 si $a_i = b_i$, et 0 sinon.

5. Si la distance naïve vaut 0, que peut-on en déduire ?

6. Recopiez et complétez la fonction suivante, pour qu'elle calcule la distance naïve entre deux chaînes. Votre fonction doit renvoyer une erreur si les deux chaînes n'ont pas la même longueur.

```
(*Calcule la distance naïve entre deux chaînes*)
let rec distance_naive (g1: genome) (g2: genome): int = match (g1, g2) with
| (x1::q1, x2::q2) -> (*à compléter*)

| ([], []) ->          (*à compléter*)

| ([], _) | (_, []) -> (*à compléter*)
```

En pratique, la distance naïve n'est pas adaptée pour comparer des génomes. En effet, lors des mutations, il est courant que des parties de génomes soient supprimées ou ajoutées.

7. Quelle est la distance naïve entre AACGTA et ACGTAC ?

III.2) Distance d'édition

On définit la **distance de Levenshtein**, ou **distance d'édition**, pour palier à ce problème.

On définit les **opérations d'édition** suivantes :

- Ajout : on ajoute un caractère.
- Suppression : on supprime un caractère.
- Modification : on change un caractère en un autre.

La **distance d'édition** entre les deux chaînes est le nombre minimal d'opérations d'édition pour passer de l'une à l'autre.

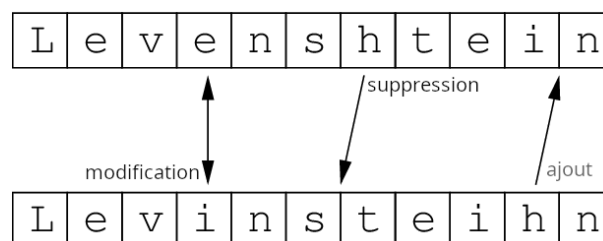


Figure 1: Distance d'édition

8. Quelle est la distance d'édition entre les deux chaînes de la figure 1 ?

9. Quelle est la distance d'édition entre la chaîne vide [] et une chaîne quelconque g ? (Pour accéder à la longueur d'une liste g, on utilise `List.length g`).

On suppose maintenant que les deux chaînes sont de longueur non nulle, et on les note $g1 = x1 : q1$ et $g2 = x2 : q2$.

10. En supposant $x1 = x2$, donnez la distance d'édition entre $g1$ et $g2$ en fonction de celle entre $q1$ et $q2$.

11*. En supposant $x1 \neq x2$, donnez la distance d'édition entre $g1$ et $g2$ avec une formule récursive. Vous considèrerez la première opération effectuée (modification, ajout, ou suppression).

12*. En déduire une fonction `distance_edition` de type `genome -> genome -> int` qui calcule la distance d'édition entre deux génomes. Votre fonction peut ne pas être optimale, tant qu'elle renvoie la bonne solution.

IV) Recherche d'un mot dans une chaîne

IV.1) Méthode naïve

On aimerait savoir si une certaine séquence d'ADN (un gène, par exemple) est présente dans le génome.

On définit le type OCaml suivant pour les gènes :

```
type gene = char list
```

À noter que ce type est identique au type `genome`. La distinction sert donc uniquement à la clarté du code.

13. Écrivez une fonction `est_prefixe` de type `genome -> gene -> bool` qui renvoie vrai si le gène est présent **au début** du génome.

14. En utilisant votre fonction `est_prefixe`, écrivez une fonction `est_facteur` de type `genome -> gene -> bool` qui renvoie vrai si le gène est présent dans le génome.

Dans les questions précédentes, l'algorithme consiste à tester toutes les positions possibles du gène dans le génome. Mais on peut faire mieux !

IV.2) Algorithme de Rabin-Karp (simplifié)

Considérons l'exemple suivant : on recherche le gène ACCT dans une chaîne.

L'astuce est de commencer la comparaison par la fin du gène. On commence donc ici par comparer la lettre T et la lettre G.

```
? ? ? G ? ? ? ? ? ? ? ...
      |
A C C T - - - - -
```

15. Sachant qu'on a vu un G à la 4ème position du génome, quelle est la première position qui pourrait accueillir le gène ? Combien de comparaisons évite-t-on alors ?

L'idée est donc de "sauter" des comparaisons lorsqu'on est sûr que le gène ne pourra pas être présent à un endroit.

On va pré-calculer, pour chaque lettre possible (parmi A, C, G, T), et pour chaque indice du gène (un entier entre 0 et `List.length gene - 1`), l'indice de décalage minimum entre deux tests (entre 1 et `List.length gene`). Par exemple, pour gène A C C T, le tableau à calculer est :

Lettre / Indice	3	2	1	0
A	3	4	4	0
C	1	0	0	4
G	4	4	4	4
T	0	4	4	4

Table 1: Table de décalage pour le motif ACCT

On note un décalage de 0 quand l'indice est à

On utilise une table de hashage, associant un couple `char * int` à un décalage. On dispose des fonctions suivantes :

- `Hashtbl.create n` pour créer une table avec `n` emplacements de départ.
- `Hashtbl.add clé valeur` pour ajouter une correspondance `clé ↔ valeur` à la table.
- `Hashtbl.find clé` renvoie la valeur associée à une clé, ou une erreur si elle est absente.

16*. Écrivez une fonction de type `gene -> ((char * int) * int) Hashtbl.t` permettant de calculer cette table. Vous pouvez simplement proposer des pistes pour réaliser la fonction sans l'implémenter.

17*. Écrivez une fonction de type `((char * int) * int) Hashtbl.t -> genome -> bool` qui renvoie vrai si le gène est présent dans le génome, en utilisant l'idée développée.